SECURITY RESEARCH BRIEF

Mật mã cổ điển - Hậu lượng tử - TLS

Có thể ép TLS 1.3 xuống TLS 1.2 không?

Tóm tắt lý thuyết | Laboratory

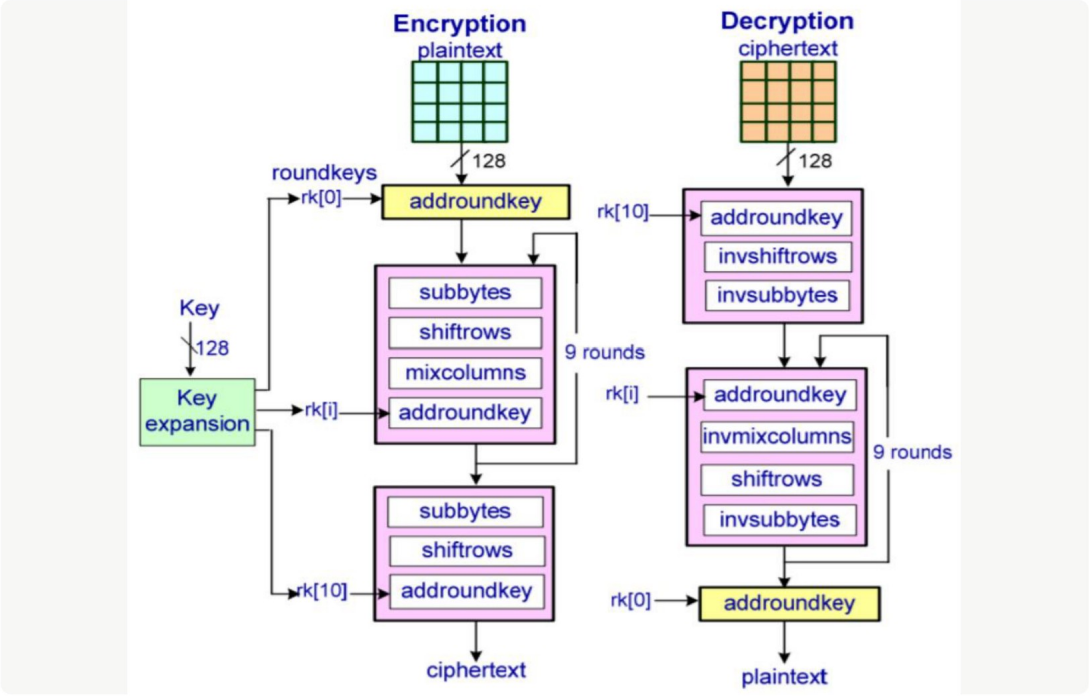
Nguyễn Hữu Duẩn

Ba nhóm thuật toán mật mã cổ điển

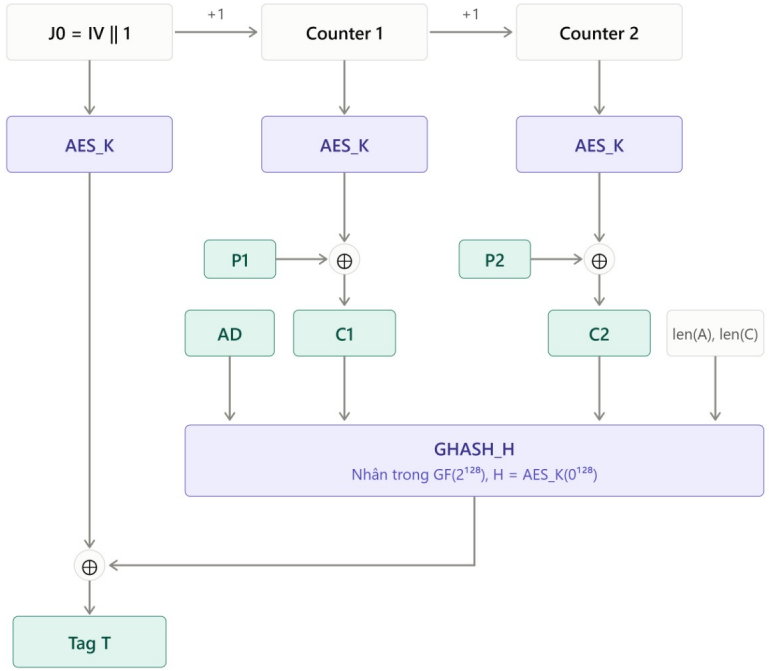
Nhóm	Thuật toán	Vai trò trong TLS
Đối xứng	AES-GCM / CCM	Mã hóa dữ liệu
Trao đổi khóa	RSA, DH / ECDH	Thiết lập bí mật chung
Chữ ký	RSA-PSS, ECDSA, EdDSA	Xác thực danh tính

AES & AES-GCM: lõi mã hóa đối xứng trong TLS

AES — mã khối Rijndael (10 round, khóa 128-bit)

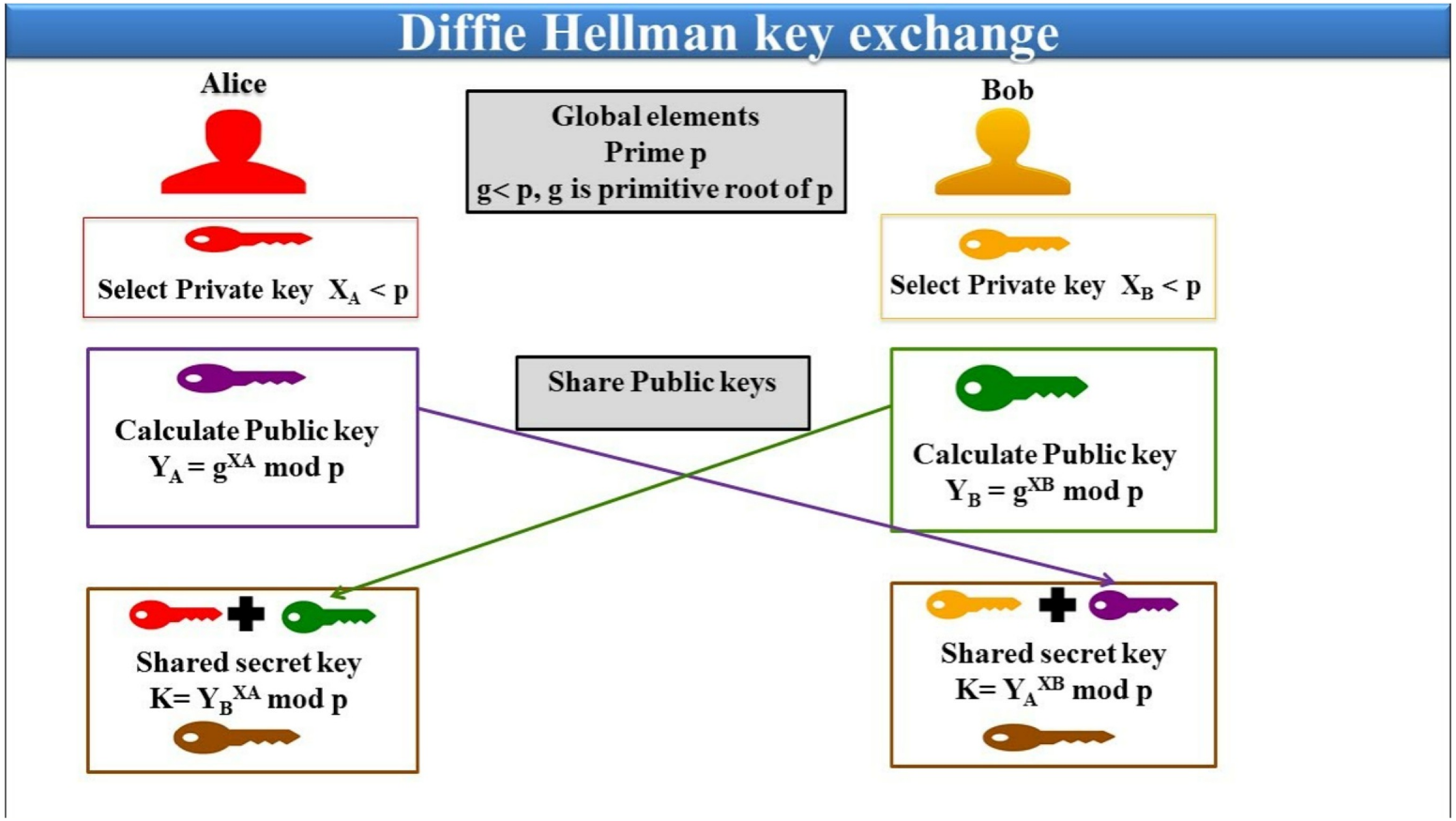


AES-GCM — CTR (mã hóa) + GHASH (xác thực) = AEAD



Cipher suite TLS dùng GCM: **TLS_AES_128_GCM_SHA256**, **TLS_AES_256_GCM_SHA384** — mã hóa & toàn vẹn dữ liệu trong một bước, không cần HMAC riêng như CBC

Trao đổi khóa Diffie–Hellman (DH/DHE)



TLS dùng biến thể đường cong elliptic — **ECDHE** (X25519, P-256…): khóa ngắn hơn, nhanh hơn DH cổ điển; "E" (Ephemeral) = khóa tạm mỗi phiên ⇒ **forward secrecy**

Chữ ký số: ký & xác thực

Ký (Sign) — bên gửi

Thông điệp M → Hash(M)

↓

Ký bằng Private Key

↓

Gửi kèm: M + Signature

Xác thực (Verify) — bên nhận

M + Signature → Hash(M)

↓

Mở khóa Signature bằng Public Key, so khớp hash

↓

Khớp ⇒ Hợp lệ | Lệch ⇒ Từ chối

Chỉ chủ **private key** ký được, ai cũng xác thực bằng **public key** ⇒ toàn vẹn + không thể chối bỏ. TLS dùng CertificateVerify để server chứng minh sở hữu private key khớp certificate.

Máy tính lượng tử phá nhóm nào?

Nhóm	Thuật toán lượng tử	Hệ quả
Đối xứng (AES)	Grover	Giảm nửa độ mạnh → tăng khóa (AES-256) là đủ
Trao đổi khóa + Chữ ký	Shor	Phá hoàn toàn → phải thay bằng PQC

"Harvest now, decrypt later" ⇒ trao đổi khóa cần chuyển PQC **trước**

PQC theo NIST: ba tiêu chuẩn 2024

2016
NIST kêu gọi đề xuất PQC

2022
Chọn Kyber, Dilithium, SPHINCS+

2022
SIKE — ứng viên vòng 4 — **bị phá**

13/08/2024
Ban hành FIPS 203 / 204 / 205

FIPS 203
ML-KEM
Trao đổi khóa (thay ECDH)

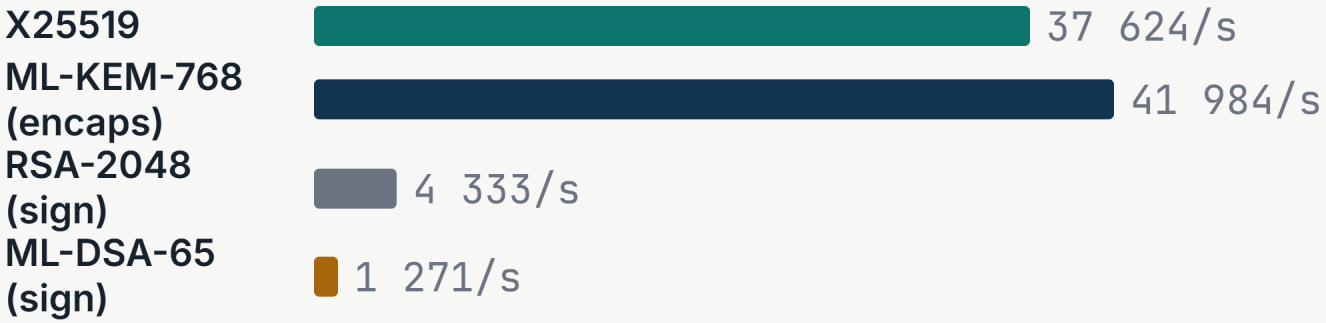
FIPS 204
ML-DSA
Chữ ký (thay RSA/ECDSA)

FIPS 205
SLH-DSA
Chữ ký dựa trên hàm băm — bảo thủ nhất

Bài học **SIKE** (bị phá 2022) ⇒ chiến lược **hybrid** cho giai đoạn chuyển tiếp.

PQC: đánh đổi kích thước & tốc độ

	X25519	ML-KEM-768	RSA-2048 (ký)	ML-DSA-65
Khóa / chữ ký	32 B	pk 1184 B	chữ ký 256 B	chữ ký 3309 B
Tốc độ (i7-11800H)	37 624 op/s	encaps 41 984/s	sign 4 333/s	sign 1 271/s



Kết luận: ML-KEM **nhanh ngang ECDH** — giá phải trả là **băng thông**.

Hybrid X25519 + ML-KEM-768 trong TLS 1.3

key_share =

ML-KEM-768 ek (1184 B) + X25519 (32 B)

An toàn kiểu "HOẶC"

Vẫn an toàn nếu **ít nhất một** thành phần còn an toàn — cổ điển **hoặc** PQC

Phòng rủi ro ML-KEM bị phá bất ngờ, như SIKE

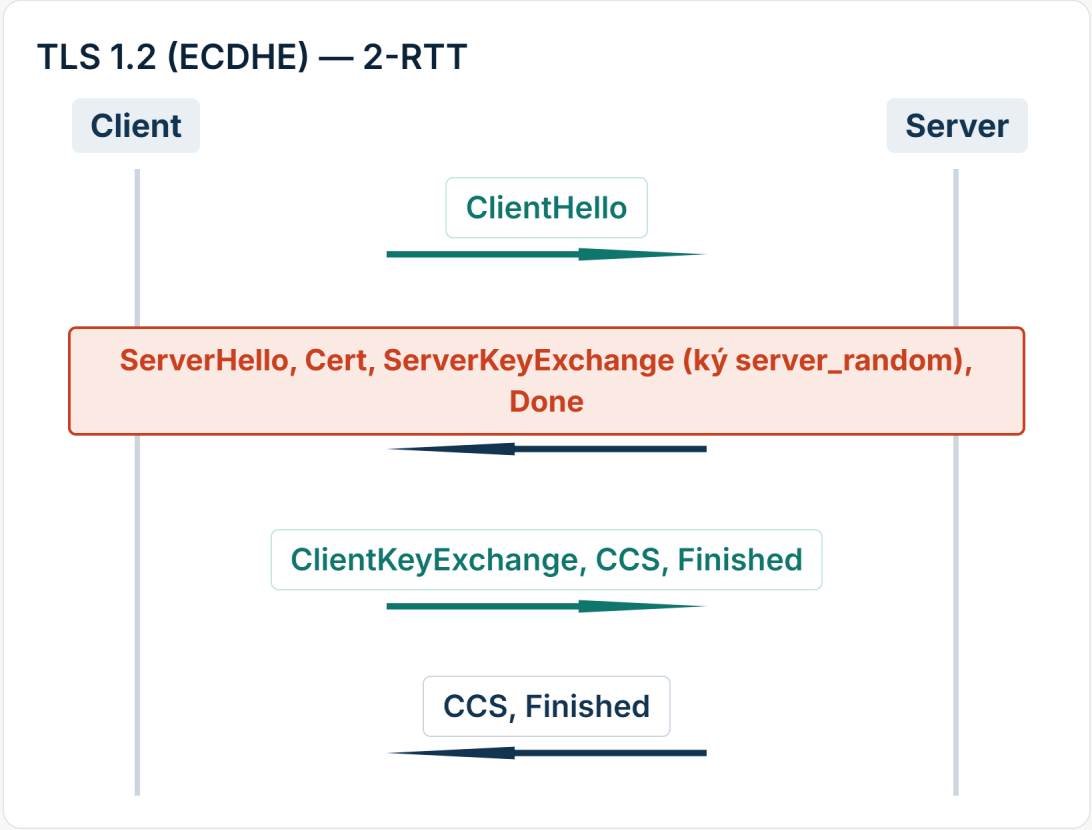
Tích hợp vào TLS

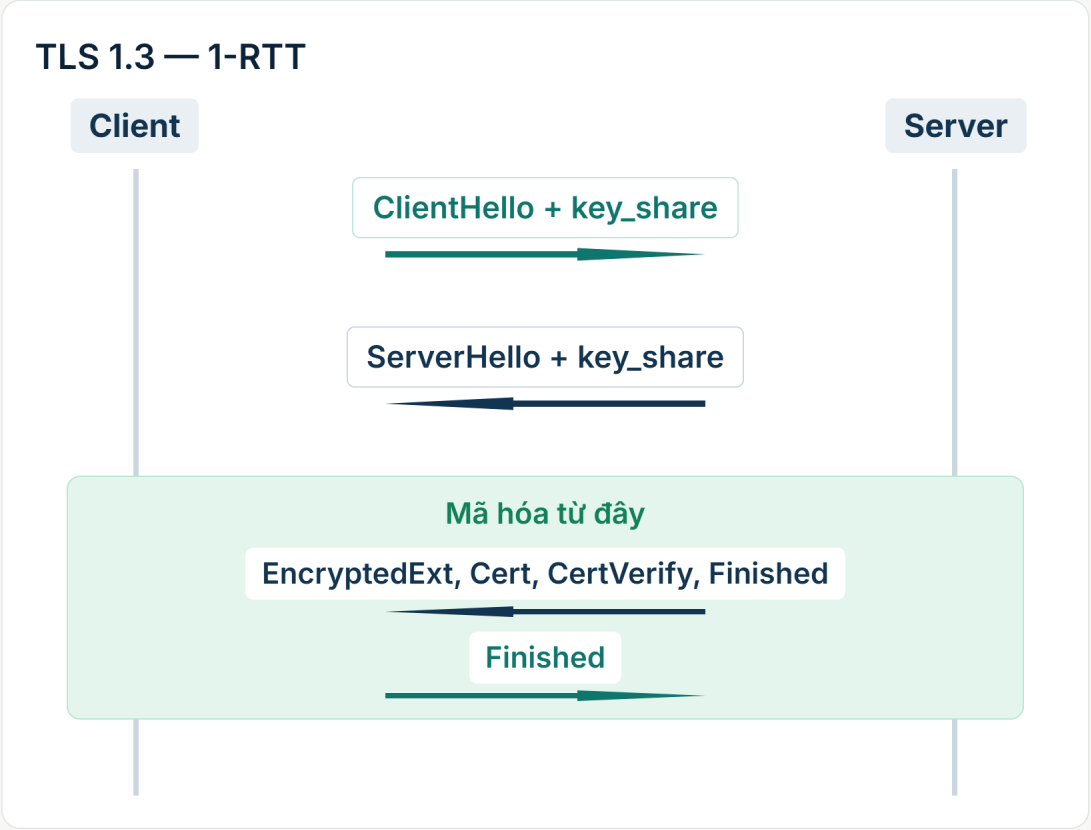
Named group mới: X25519MLKEM768 = 0x11EC

Không đổi cấu trúc handshake — chỉ thêm một key_share

Bật mặc định: Chrome/Firefox (2024), OpenSSL 3.5, Go 1.24+

TLS 1.2 vs TLS 1.3: sơ đồ bắt tay





⚠ Lịch sử tấn công TLS 1.2 — động lực của 1.3

Logjam — 2015

Một **tấn công hạ cấp**: ép DH export 512-bit → phá được

⇒ động lực trực tiếp cho sentinel chống hạ cấp

Lucky Thirteen — 2013

Padding oracle qua **timing side-channel** trên đệm CBC

⇒ TLS 1.3 bắt buộc AEAD, bỏ CBC

⚡ Điểm chung: giữ thuật toán cũ + cho phép thương lượng xuống = **nguy hiểm**

Cơ chế chống hạ phiên bản của TLS 1.3

server_random (32 byte) khi server buộc phải chọn 1.2

24 byte ngẫu nhiên	44	4F	57	4E	47	52	44	01

8 byte cuối = "DOWNGRD" + 0x01 — client TLS 1.3 PHẢI từ chối khi thấy mẫu này

1. Thương lượng qua extension supported_versions — không dùng legacy_version
2. Sentinel **được chữ ký của server bảo vệ** — MITM không có khóa riêng để sửa
3. TLS_FALLBACK_SCSV chống fallback ở tầng ứng dụng

Server luôn đặt sentinel | **Chữ ký phủ random** | **Client bắt buộc kiểm tra**

Demo lab: môi trường & kịch bản



`OpenSSL 3.5.5` `Go 1.26.8` `Python MITM` `tcpdump` `CA nội bộ`

8 kịch bản kiểm chứng

S1 baseline · **S2** strip supported_versions · **S2b** lộ sentinel · **S3** xóa sentinel → chữ ký bắt · **S4** xóa sentinel → Finished bắt

S5 client tự fallback · **S5b** SCSV chặn · **S6** bỏ verify chứng chỉ

Demo lab

Kịch bản	MITM làm gì	Kết quả thật	Ý nghĩa
S1	Không can thiệp — chạy thẳng để lấy baseline	TLSv1.3 (+ hybrid X25519MLKEM768)	baseline, không sentinel
S2	Strip extension supported_versions khỏi ClientHello	inappropriate fallback (hủy)	lớp 1–2 chặn
S2b	Ép chọn TLS 1.2 nhưng giữ nguyên sentinel DOWNGRD trong server_random	server_random … 44 4f 57 4e 47 52 44 01	sentinel thật
S3	Xóa/giả 8 byte sentinel trong server_random	bad signature → Alert decrypt_error (51)	xóa sentinel ⇒ chữ ký bắt
S4	Giả sentinel + ép cipher cũ (static RSA)	bad record mac (Alert 20)	static RSA ⇒ Finished bắt
S5	Không làm gì ở tầng TLS — chờ client tự fallback	TLSv1.2 (vẫn có sentinel, client không kiểm)	client tự fallback
S5b	Giống S5, nhưng client có gửi TLS_FALLBACK_SCSV	alert number 86	SCSV chặn
S6	Tự ký chứng chỉ giả (self-signed) cho server	verify on → certificate verify failed; off → TLSv1.2	thiếu xác thực cert

Kết luận & khuyến nghị phòng chống

Trả lời: KHÔNG hạ được khi đúng chuẩn + verify nghiêm ngặt. Mọi điều kiện làm CÓ đều NGOÀI CHUẨN.

Client

Verify chuỗi chứng chỉ + hostname nghiêm ngặt

Không tự fallback — hoặc gửi TLS_FALLBACK_SCSV

Đặt min = TLS 1.3 khi hạ tầng cho phép

Luôn cập nhật thư viện TLS

Server

Bật TLS 1.3; với 1.2 chỉ ECDHE + AEAD

Tắt TLS_RSA_* / CBC / RC4; hỗ trợ SCSV

Bật Extended Master Secret (EMS)

Giữ khóa riêng trong HSM

Cảm ơn hội đồng đã lắng nghe!

Ép TLS 1.3 xuống TLS 1.2?

Trả lời: KHÔNG, khi đúng chuẩn

Giải mã ServerHello (S2b) từng byte

Trường	Giá trị quan sát	Ý nghĩa
type	0x02	Handshake message = ServerHello
legacy_version	0x0303 (TLS 1.2)	Luôn giả định — version thật nằm ở extension
random (32B)	...[24B]||44 4f 57 4e 47 52 44 01	8 byte cuối = sentinel "DOWNGRD"+0x01
cipher_suite	0xC030	ECDHE-RSA-AES256-GCM-SHA384
extensions	thiếu ext 43 (supported_versions)	Server không công bố 1.3 ⇒ giống ServerHello 1.2 thật, chỉ sentinel tố cáo

⚠ Các nước đi của MITM & vì sao đều hủy

#	Nước đi của MITM	Vì sao hủy / vì sao thành công
1	Strip extension supported_versions khỏi ClientHello	Server vẫn có thể trả lời bằng 1.3 thật nếu hỗ trợ — chỉ ẩn khả năng client, không tự hạ cấp
2	Ép server chọn 1.2 khi cả hai hỗ trợ 1.3	Lộ sentinel DOWNGRD trong server_random — client PHẢI từ chối nếu kiểm tra
3	Xóa/giả 8 byte sentinel trong server_random	Chữ ký ServerKeyExchange không còn khớp random đã sửa → bad signature
4	Ép cipher cũ (static RSA) sau khi giả sentinel	Finished/MAC không khớp transcript thật → bad record mac
5	Không làm gì ở tầng TLS — chỉ chờ client tự fallback khi lỗi	Thành công NẾU client fallback mà không gửi TLS_FALLBACK_SCSV
6	Tự ký chứng chỉ giả (self-signed) cho server	Thành công NẾU client không xác thực chuỗi CA nghiêm ngặt

Vì sao thiếu SAN không phải lỗi phiên bản

OpenSSL s_client

PASS — vẫn chấp nhận xác thực qua **Common Name (CN)** để tương thích chứng chỉ cũ

Go crypto/x509

FAIL — từ Go 1.15 đã **bỏ hoàn toàn fallback CN**, bắt buộc chứng chỉ có SAN

Đây là khác biệt **thư viện xác thực chứng chỉ** — không liên quan TLS 1.2 vs 1.3

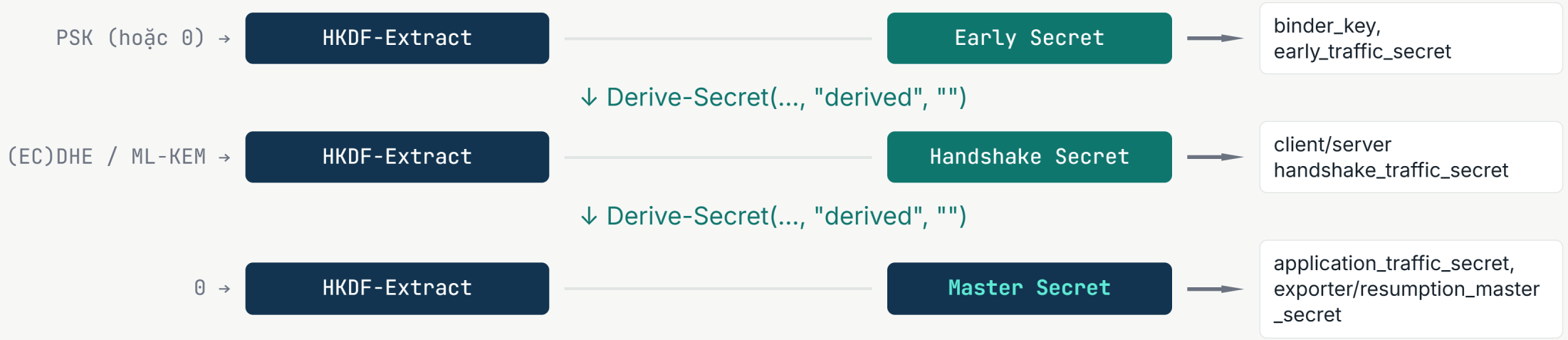
Khuyến nghị: luôn cấp chứng chỉ có SAN đầy đủ (subjectAltName), không dựa vào CN.

So sánh thư viện: hỗ trợ TLS 1.3 + PQC

Thư viện	TLS 1.3	ML-KEM	Hybrid (mặc định?)	ML-DSA
OpenSSL 3.5	✓	✓	✓	⚠
BoringSSL	✓	✓	✓	⚠
Go crypto/tls 1.24+	✓	✓	✓	✗
rustls	✓	✓	⚠	⚠
wolfSSL	✓	⚠	⚠	⚠
mbedTLS	✓	⚠	✗	✗
Java JSSE	✓	⚠	✗	⚠

⚠ = qua provider/flag bên thứ ba hoặc chưa mặc định — kiểm tra lại release notes đúng phiên bản đang dùng trong lab (slide 13).

Key schedule TLS 1.3 (HKDF)



Mọi secret dẫn xuất qua HKDF-Expand (RFC 5869) — thay cho PRF của TLS 1.2 (slide 10).